



7.4 逻辑设计

逻辑设计和逻辑模型

- 逻辑设计

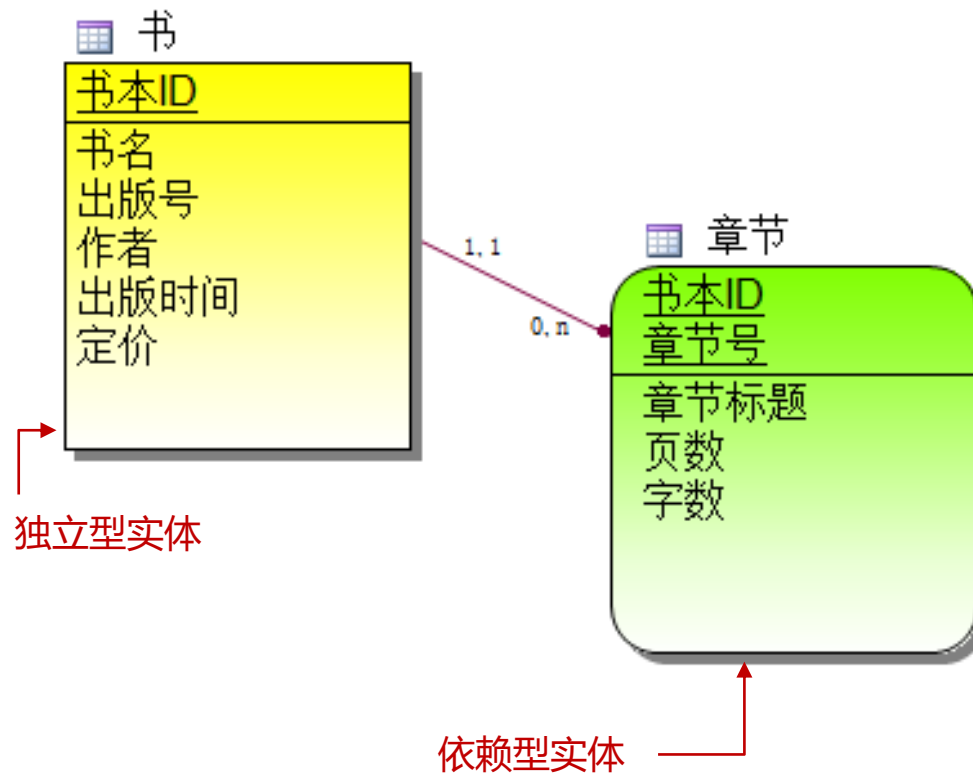
- 这个阶段是将概念模型转化为具体的数据模型的过程。
- 按照概念设计阶段建立的基本E-R图，按选定的目标数据模型（层次、网状、关系、面向对象），转换成相应的逻辑模型。
- 对于关系型数据库来说，这种转换要符合关系数据模型的原则，得到的就是逻辑数据模型。
- 这个阶段主要的工作就是确定关系模型里面的属性和码（或者说主键）。
- 比较常用的方式是使用E-R设计工具，IDEF1x方法来进行逻辑模型建设，常用的ER图表示法包括IDEF1x，IE模型的Crow's foot，UML类图方式等。

IDEF1x方法

- IDEF1X(Integration DEFinition for Information Modeling)
 - 信息模型集成定义。
 - IDEF1X是IDEF系列方法中IDEF1的扩展版本，是在E-R（实体联系）方法的原则基础上，增加了一些规则，使语义更为丰富的一种方法。
- IDEF1X特点：
 - 良好的可扩展性。
 - 简明的一致性结构。
 - 便于理解。
 - 可以自动化生成模型。

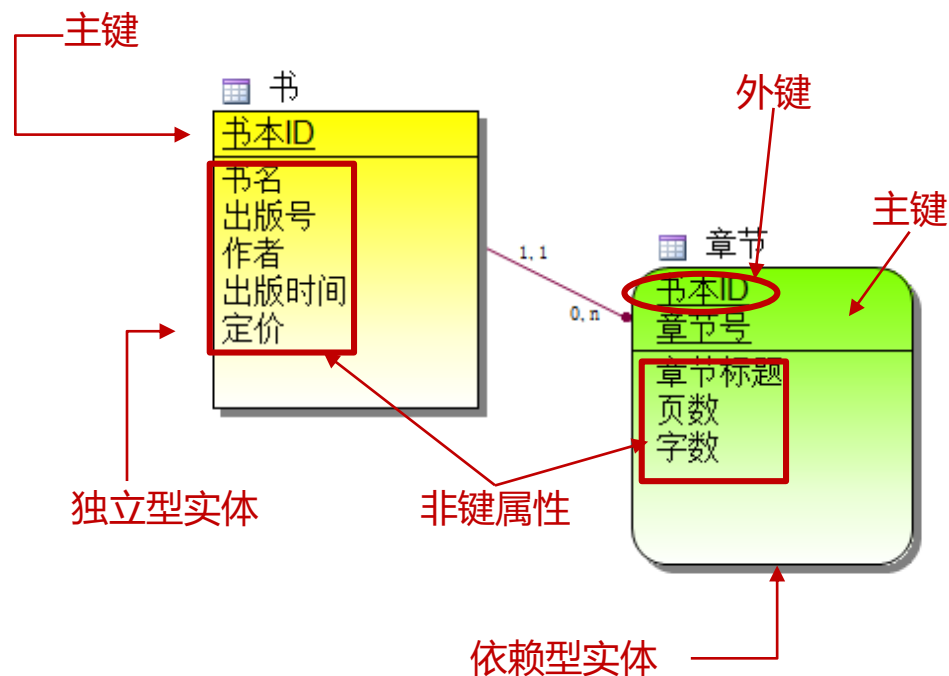
逻辑模型中的实体

- 根据实体的特点，划分为两类：
 - 独立型实体(Independent Entity)
 - 直角矩形表示。
 - 不依赖于其他实体，可以独立存在。
 - 依赖型实体(Dependent Entity)
 - 圆角矩形表示。
 - 必须依赖于其它实体而存在。
 - 依赖型实体中的主键必须是独立实体主键的一部分或者全部。



实体中的属性

- 属性是实体的特征，需要注意的类型有：
 - 主键(Primary Key)
 - 识别实体实例唯一性的属性或属性组。
 - 可选键
 - 能识别实体实例唯一性的其他属性或者属性组。
 - 外键
 - 两个实体发生关联，一个实体的外键是另外一个实体的主键。
 - 也可以把主键实体称为父实体，拥有外键的实体称为子实体。
 - 非键属性
 - 实体里面除主键和外键属性外的其他属性。
 - 派生属性
 - 一个字段可以被统计出来或者从其它字段推导出来的字段。



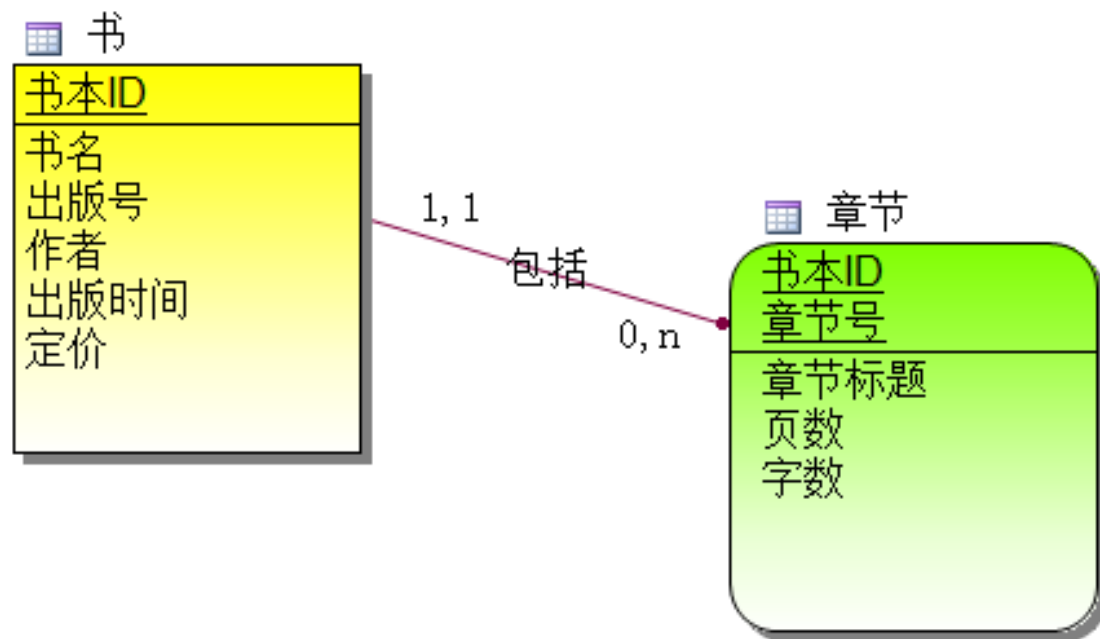
主键，外键和索引之间的关系

- 主键、外键、索引之间的联系如下表：

	主键	外键	唯一索引	非唯一索引
特点	唯一标识一个实例，无重复值、非空。不应该被更新	另外一个实体的主键，可以重复，可以为空。	建立在表上的对象。无重复值，可以有一个空值	建立在表上的对象，可以为空，也可以重复值
作用	确定记录唯一性，保证数据完整性。	建立数据参考一致性。建立两个实体之间的关系。	提高查询效率	提高查询效率
数量	一个实体只能有一个	一个实体可以有多个外键	一个表可以有多个唯一索引	一个表可以有多个非唯一索引

实体间的关系

- 关系是描述实体间如何发生关联
 - 一本书“包括”若干个章节。
 - “包括”就是这两个实体之间的关系。
 - 关系是有方向性的。
- 关系基数(Cardinality)
 - 反映两个或多个实体间关系的业务规则。



IDEF1x中基数的图例



0,1或多个



1或多个



0或多个



有且只有n

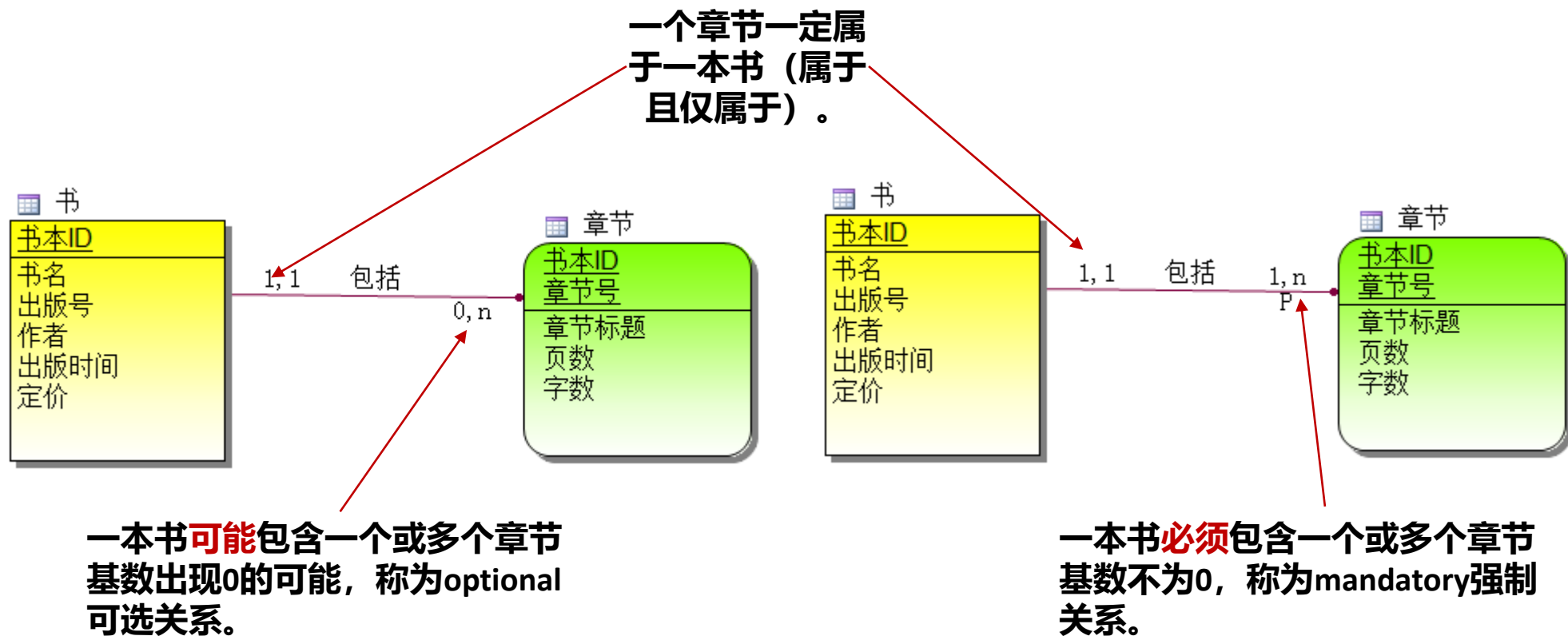


n-m个



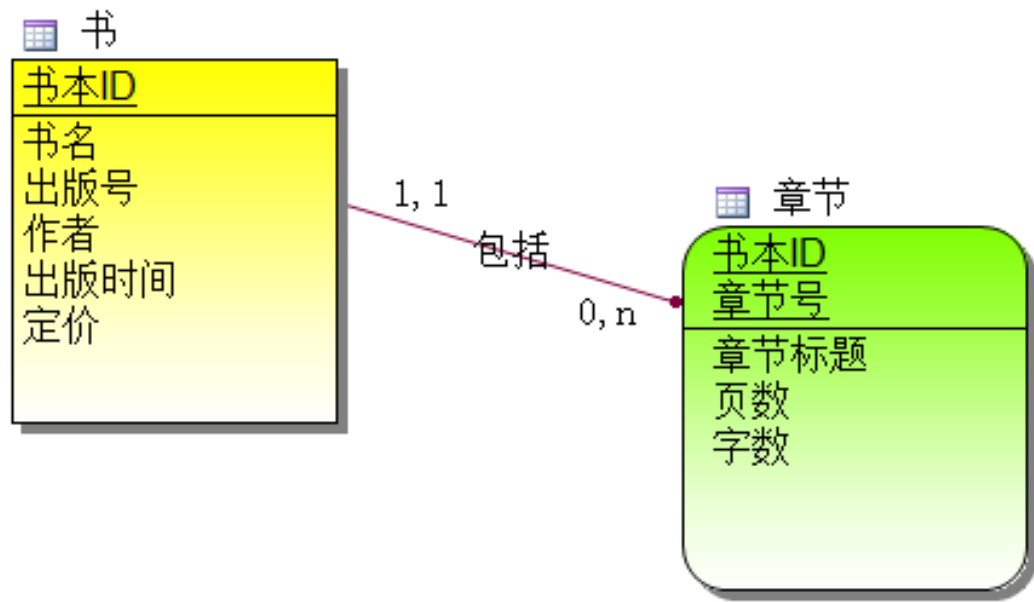
参考相关注释,
基数在注释中
说明

基数的不同反映了不同的关系



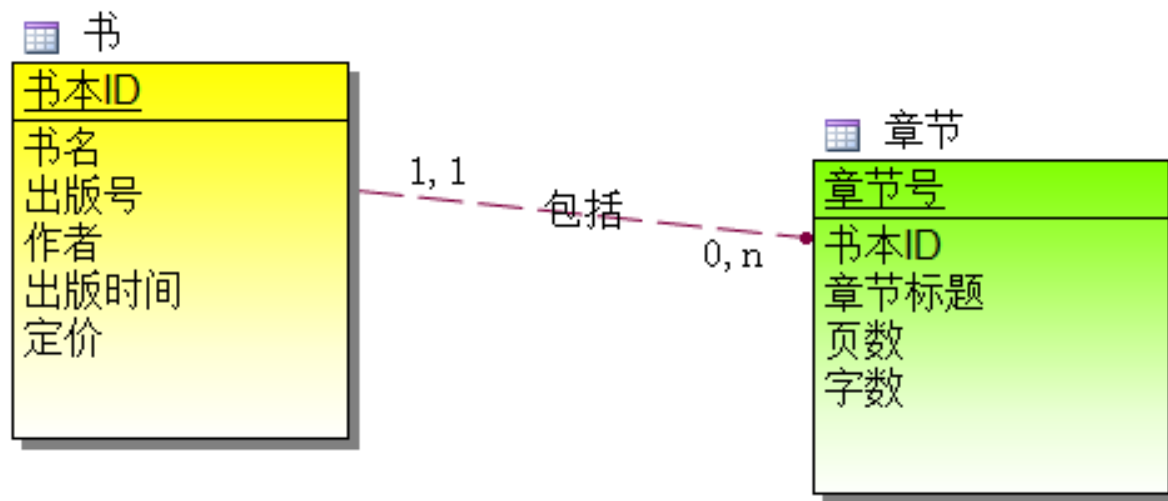
识别性关系

- 识别性关系(Identifying relationship)
 - 发生在独立型实体和依赖型实体之间。
 - 子实体的实例唯一性的识别与父实体相关联。
 - 父实体的主键属性成为子实体的主键属性。



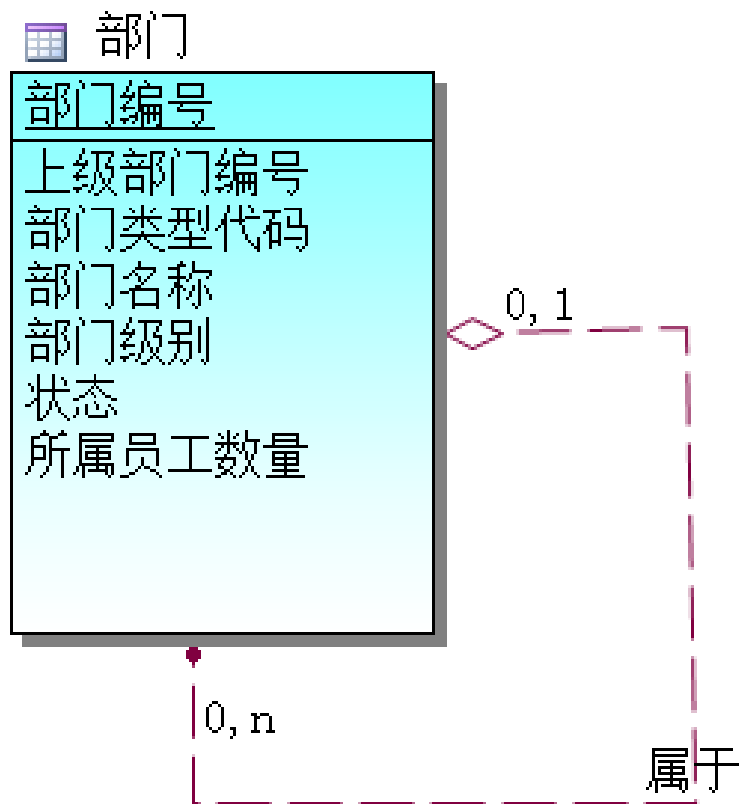
非识别性关系

- 非识别性关系(non-identifying relationship)
 - 子实体不需要与父实体的关系就可以确定实例唯一性。



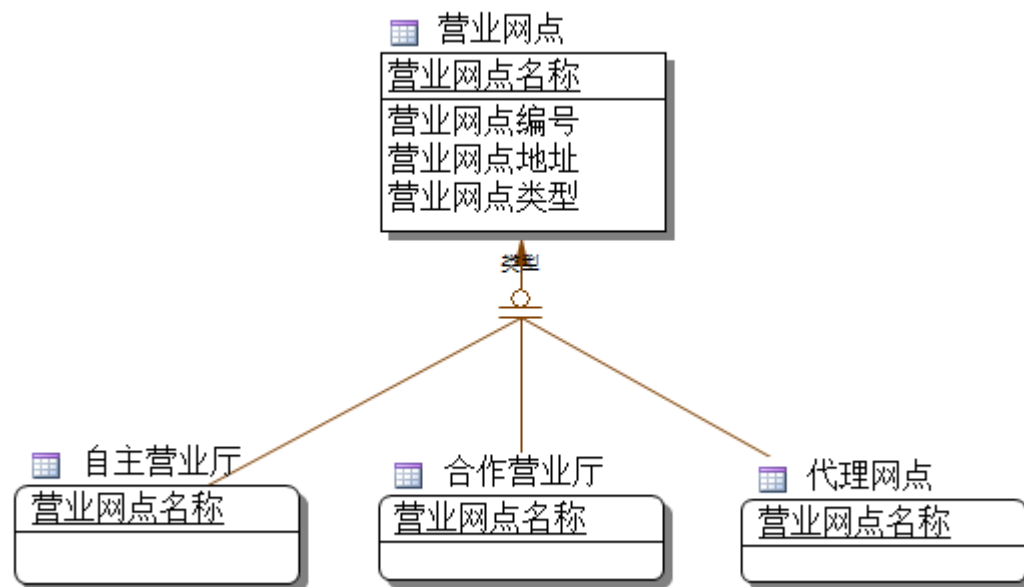
嵌套关系

- 嵌套关系(Recursive relationship)
 - 父实体和子实体为同一个实体，形成递归或者嵌套的关系。
 - 实体的主键也成为自身的外键。



子类关系

- 子类关系(Subtype relationship)
 - 子类实体和所属父实体的关系。
 - 完全子类关系。
 - 所属父实体的每个实例都能够与子类群的一个实体实例相关联。
 - 不完全子类关系。
 - 所属父实体的每个实例不一定都与子类群



逻辑模型基本概念小结

- 实体就是描述业务的元数据。
- 主键是识别实体每一个实例唯一性的标识。
- 只有存在外键，实体之间才会存在关系，没有外键不能建立关系。
- 关系的基数反映了关系之间的业务规则。
 - 一个客户只能拥有一个一类储蓄账户。
 - 一个客户可以拥有多个储蓄账户。
 - 一个订单只能对应一个发货运单。
 - 一个产品包括多个零件。

范式理论 (Normal Form)

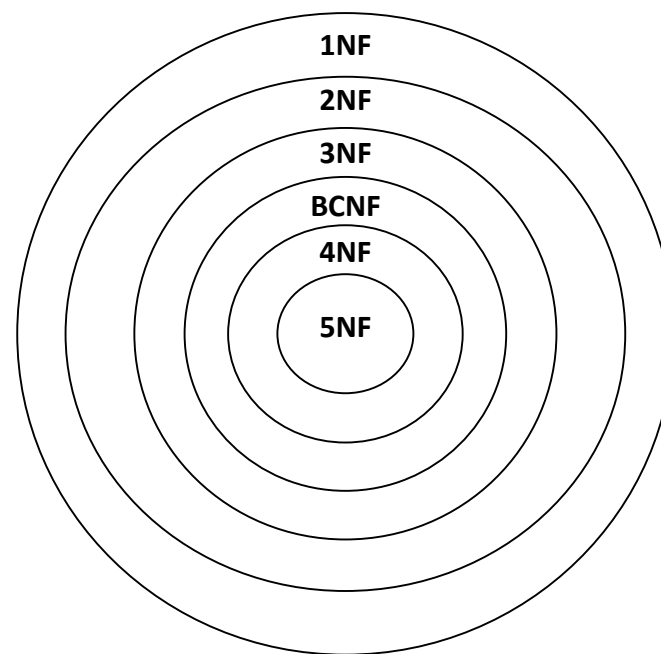
- 在数据库设计的时候要满足的设计规范
 - 满足不同程度的要求为满足不同的范式;
 - 把属性放置在正确的实体的这个过程称为范式化(normalization).
- 发展历史
 - 1971~1972: Codd系统地提出了1NF、2NF和3NF的概念, 讨论了规范化问题;
 - 1974: Codd和Boyce共同提出了新范式, BCNF;
 - 1976: Fagin提出了4NF;
 - 之后的研究人员进一步提出5NF。

范式理论的意义

- 设计逻辑模型时候最常遇到的问题
 - 哪些属性应当放到实体中，怎么放？
- 范式化的意义：
 - 减少数据冗余。
 - 提供良好的可扩展性。
 - 消除数据更新时候可能产生的数据不一致情况。

范式之间的关系

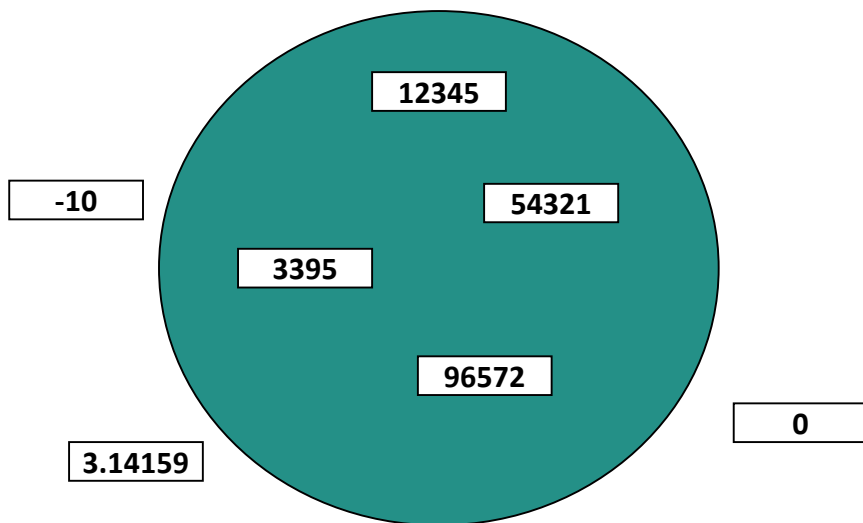
- 范式之间的关系：
 - 满足最低要求的叫第一范式，记为1NF。
 - 在第一范式满足进一步要求的为第二范式，2NF。
 - 以此类推。
 - 一个低一级范式的关系模式通过模式分解 (Schema Decomposition) 可以转换为若干个高一级范式的关系模式的集合。



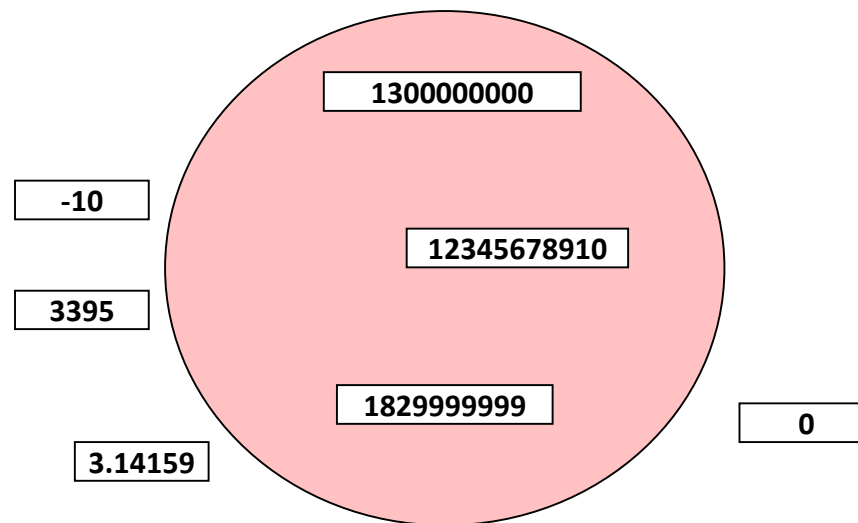
值域 (Domain)

- 定义一个属性取值的有效范围
 - 在值域里面的值都是合法数据。
 - 值域体现了规则。

员工号码，大于0的整数



手机号码，11位长度的整数



第一范式 (1NF) (1)

- 第一范式
 - 属性取值的原子性（不可再分）。
 - 属性取值数量是单一的，不能是值域里面的子集。
 - 需要有主键。
 - 实体中的属性不存在重复组问题。

客户表

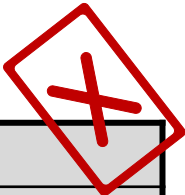
客户id	姓名	年龄	电话号码
PK			
123	XXX	30	555-666-1234, 333-888-5678
456	YYY	40	555-777-8080转43, 155-0099-9900
789	ZZZ	50	777-808-9234

出现的取值数量多于一
个，不具备原子性

第一范式 (1NF) (2)

● 客户表

两个“电话号码”的属性构成了
“repeating group”



客户id	姓名	年龄	电话号码1	电话号码2
PK				
123	XXX	30	555-666-1234	333-888-5678
456	YYY	40	555-777-8080转43	155-0099-9900
789	ZZZ	50	777-808-9234	



第一范式
符合

客户id	姓名	年龄	电话号码
PK			
123	XXX	30	555-666-1234
123	XXX	30	333-888-5678
456	YYY	40	555-777-8080转43
456	YYY	40	155-0099-9900
789	ZZZ	50	777-808-9234

“原子性” 的程度

- 原子性概念在实际应用中容易出现歧义。
 - 不可再分的程度如何确定边界。
 - 具有编码规则的代码实际上都是复合型代码，从规则上讲都是可分的。
 - 比如身份证号码、手机号码，都是可以进一步拆分出更细粒度的数据。
- 从值域的角度来理解。
 - 身份证号码的值域：只要符合身份证编码规则的就是合法的身份证，就是原子性的数据。

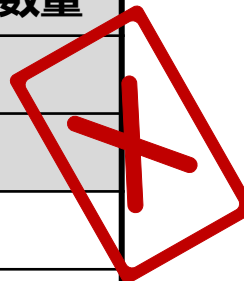
第二范式 (2NF) (1)

- 第二范式的两个必要条件：
 - 首先要满足第一范式。
 - 每一个非主属性都完全函数依赖于任何一个主键。

订单零件表

订单编号	零件编号	订单日期	所需零件数量
PK			
FK	FK		
1000	1234	2010-08-01	200
1000	5678	2010-08-01	100
2000	1234	2010-11-15	50
3000	7890	2010-09-30	300

订单日期只依赖于订单编号，和零件编号无关



第二范式 (2NF) (2)

- 修改办法：

订单零件表

订单编号	零件编号	所需零件数量
PK		
FK	FK	
1000	1234	200
1000	5678	100
2000	1234	50
3000	7890	300



订单编号	订单日期
PK	
1000	2010-08-01
2000	2010-11-15
3000	2010-09-30

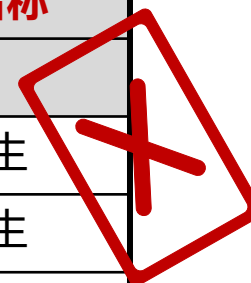
第三范式 (3NF) (1)

- 第三范式的两个必要条件：
 - 首先要满足第二范式。
 - 每一个非主属性不会传递性依赖于主键。

订单客户表

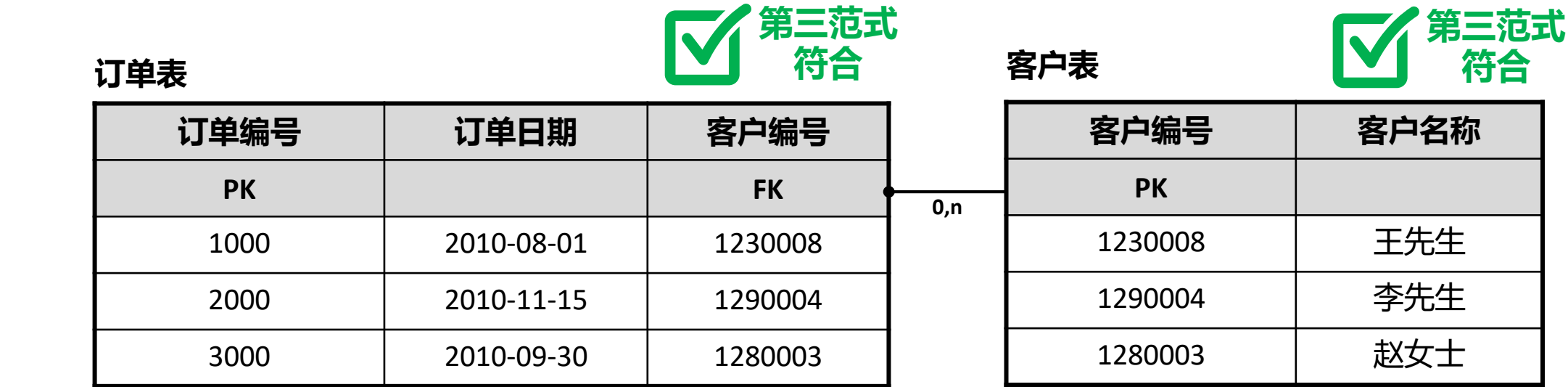
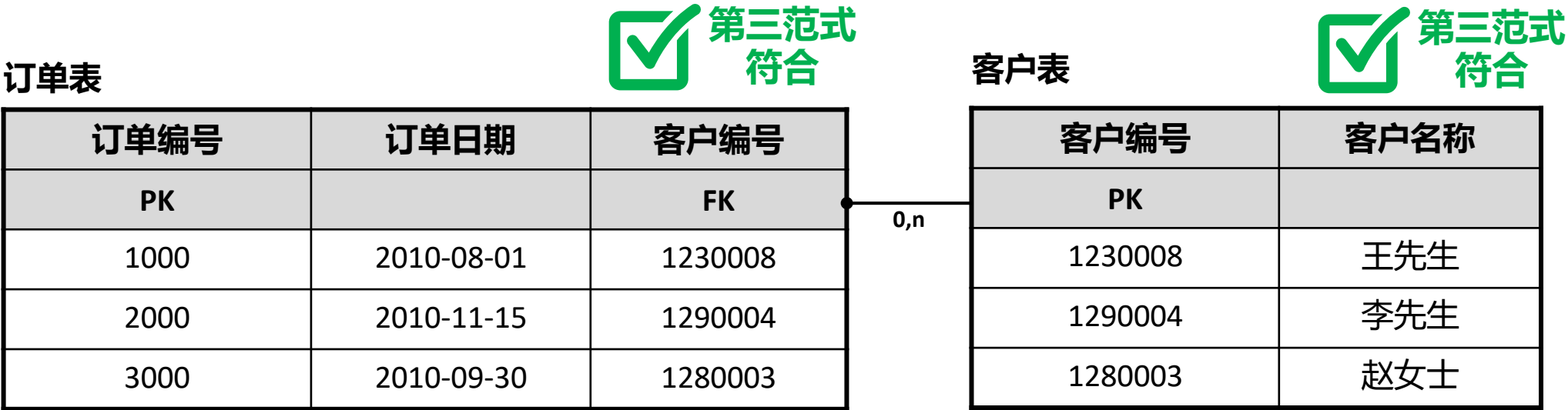
订单编号	订单日期	客户编号	客户名称
PK			
1000	2010-08-01	1230008	王先生
2000	2010-11-15	1290004	李先生
3000	2010-09-30	1280003	赵女士

客户名称依赖于非主键
属性“客户编号”



第三范式 (3NF) (2)

- 拆表，利用外键形成关系



一句话精炼

The **KEY** – 1st Normal Form (1NF)

The **WHOLE** Key – 2nd Normal Form (2NF)

And **NOTHING BUT** the Key – 3rd Normal Form (3NF)

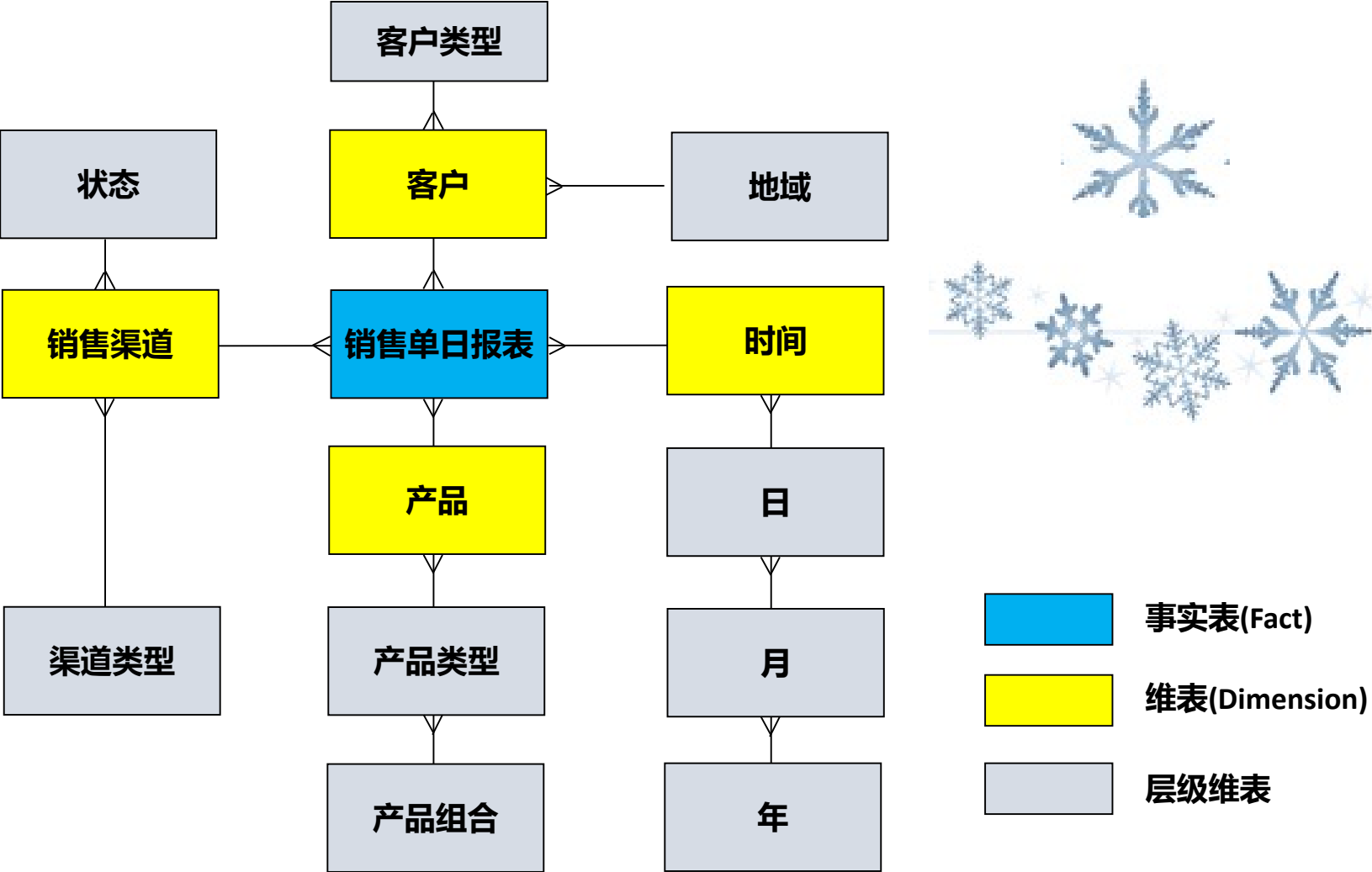
--

1.要有主键;

2.依赖于整个主键;

3.只能依赖于主键。

雪花型模型



逻辑模型建设注意事项 (1)

- 建立命名规则
 - 命名规则的意义：
 - 统一命名，避免歧义。
 - 防止冗余的实体或者属性产生。
 - 有利于工作中不同角色的人员之间通过规范的命名和属于进行交流。
 - 便于使用。
 - 实体和属性的命名建议：
 - 实体名称：分类域大写+实体描述词（全称，首字母大写）。
 - 属性名称：使用全称，首字母大写，一些约定俗称的空格缩写。
 - 避免英语和拼音的混用。
 - 如果是缩写，一定是英语的缩写，避免使用拼音的声母缩写。

逻辑模型建设注意事项 (2)

- 按照设计流程设计逻辑数据模型
- 确定实体和属性：
 - 定义实体的主键(PK).
 - 定义部分非键属性(Non-Key Attribute).
 - 定义非唯一属性组。
 - 添加相应的注释内容。

逻辑模型建设注意事项 (3)

- 确定实体与实体之间的关系：
 - 通过外键来体现。
 - 决定实体之间是否是可识别的关系。
 - 确定关系的基数属于1:1, 1:n还是n:m。
- 补充实体的非键值属性：
 - 按照3NF的规则, 判定添加的属性是否符合3NF的设计原则。
 - 如果新增属性违反3NF, 需要进行实体拆分, 确定新的实体和关系。
 - 添加注释。